

# CUT&RUN analysis pipeline

Evan Hass

2026-01-09

## 1. Adapter trimming with trimmomatic

```
module load trimmomatic/0.32

java -jar /pi/thomas.fazzio-umw/conda/pkgs/trimmomatic-0.39-hdfd78af_2/share/trimmomatic-0.39-2/trimmomatic.jar PE \
  SAMPLE_R1.fastq.gz \
  SAMPLE_R2.fastq.gz \
  SAMPLE_trim_1_paired.fq.gz \
  SAMPLE_trim_1_unpaired.fq.gz \
  SAMPLE_trim_2_paired.fq.gz \
  SAMPLE_trim_2_unpaired.fq.gz \
  ILLUMINACLIP:/pi/thomas.fazzio-umw/conda/pkgs/trimmomatic-0.39-hdfd78af_2/share/trimmomatic-0.39-2/adapters/TruSeq3-PE-2.fa:2:25:7:1:true TRAILING:3 SLIDINGWINDOW:4:10 TRAILING:3 MINLEN:10
```

Breaking down the command:

- `java -jar /pi/thomas.fazzio-umw/conda/pkgs/trimmomatic-0.39-hdfd78af_2/share/trimmomatic-0.39-2/trimmomatic.jar` just tells the computer where to look for the `.jar` file for the trimmomatic program.
- `PE` tells the program that you're working with paired-end reads.
- Next, the command looks for your read 1 input file and read 2 input file, then you specify the file names for your trimmed reads. In the process of trimming your reads, some read pairs may have one read survive trimming but not the other, so for both read 1 and read 2, you'll need to specify file names for trimmed paired reads and trimmed unpaired reads.
- `ILLUMINACLIP` is trimmomatic's adapter trimming module. First, it looks for a fasta file specifying the adapter sequences you need to trim. This is followed by a string of six arguments separated by colons. The first five are numeric arguments that specify various thresholds for things like number of mismatches allowed between the adapter sequence and a read or minimum adapter length that can be detected. The sixth is a true/false argument specifying whether to keep both reads in a pair when both have had their adapters trimmed. See the trimmomatic manual for more details on these.
- `TRAILING` cuts bases off the end of a read if below a specified minimum quality. Full disclosure, I don't know why it's specified twice; I originally got this code from Kai in Julie's group and haven't modified it since.
- `SLIDINGWINDOW` scans the reads in a sliding window of defined size and trims the read if the average quality within that window falls below a specified threshold.
- `MINLEN` specifies the minimum length that a read must be after trimming to be retained.

## 2. FASTQC

```
module load fastqc/0.11.9

fastqc SAMPLE_trim_1_paired.fq.gz -o .
fastqc SAMPLE_trim_2_paired.fq.gz -o .
```

This command is pretty simple. It takes the fastq files you specify and outputs an html file summarizing various QC metrics for your reads. The `-o` argument is where you specify an output directory, and the `.` specifies the current working directory.

## 3. Alignment with bowtie2

```
module load bowtie2/2.5.0
module load samtools/1.16.1

bowtie2 -q -N 1 -X 1000 -p 16 -x /share/GHPCC/data/umw_biocore/genome_data/human/hg38/hg38 -1 SAMPLE_trim_1_paired.fq.gz -2 SAMPLE_trim_2_paired.fq.gz -S SAMPLE_bowtieoutput.sam
```

Breaking down the command:

- `-q` tells the program that your input files are in fastq format
- `-N` specifies the number of mismatches to be allowed during a process that bowtie2 uses called multiseed alignment. See manual for more details.
- `-X` specifies the maximum allowed length for an insert. Put another way, this is the maximum distance to allow between the sites where the 5' ends of two paired reads align.
- `-p` specifies the number of cores or "threads" to use in parallel when running this command.
- `-x` specifies the directory where the reference genome files are located. When aligning reads from the *Drosophila* spike-in, this should be specified as `/share/GHPCC/data/umw_biocore/genome_data/d_melanogaster/dm6`.
- `-S` specifies the name of the output sam file.

## 4. Removing PCR duplicates with picard

```
module load samtools/1.16.1

java -Xmx4g -jar /pi/thomas.fazzio-umw/Sarah/picard-tools-2.5.0/picard.jar \
  SortSam \
  INPUT=SAMPLE_bowtieoutput.sam \
  OUTPUT=SAMPLE_PicardSort.bam \
  VALIDATION_STRINGENCY=LENIENT \
  TMP_DIR=/tmp \
  SORT_ORDER=coordinate
java -Xmx4g -jar /pi/thomas.fazzio-umw/Sarah/picard-tools-2.5.0/picard.jar \
  MarkDuplicates \
  INPUT=SAMPLE_PicardSort.bam \
  OUTPUT=SAMPLE_PicardDedup.bam \
  VALIDATION_STRINGENCY=LENIENT \
  TMP_DIR=/tmp \
  METRICS_FILE=SAMPLE_dup.txt \
  REMOVE_DUPLICATES=true
```

This step includes two different commands, each of which use picard, which is called with `java -Xmx4g -jar /pi/thomas.fazzio-umw/Sarah/picard-tools-2.5.0/picard.jar`.

The first step uses picard's `SortSam` function to sort your aligned reads. `SORT_ORDER=coordinate` specifies that the reads will be sorted by genome coordinate.

The second step uses picard's `MarkDuplicates` function to find PCR duplicate reads. The `REMOVE_DUPLICATES=true` argument specifies that these duplicate reads will not be printed to the output file. The `METRICS_FILE` argument specifies the name of a file that will show several metrics from the deduplication process (e.g., percent duplicates).

## 5. Removing reads with low mapping quality using samtools

```
module load samtools/1.16.1

samtools view -hq 10 SAMPLE_PicardDedup.bam > SAMPLE_PicardDedup_mapqfilt.sam
```

Breaking down the command:

- `samtools view` is a very versatile command that can be used to do a lot of different things depending on the arguments you give it.
- `-hq 10` is two different options smooshed together. Given separately, it would look like `-h -q 10`. `-h` specifies that the header of the input file will be included in the output file `-q 10` specifies that any alignments with a mapping quality (MAPQ) less than 10 will be removed.
- Next, the input file is called. Technically, that's the end of the `samtools` command itself, but if we ended the line of code there, the output of this command would just be printed to the report file for your bash script. In order to print the output of this command to a new file of its own, we use `>` followed by the name of the file where we'll print the output to.

## 6. Filtering by fragments 120 bp and shorter (only necessary

## for TFs)

```
module load samtools/1.16.1

samtools view SAMPLE_PicardDedup_mapqfilt.sam |\
awk ' $9 <= 120 && $9 >= 1 || $9 >= -120 && $9 <= -1 ' > SAMPLE_mapqfilt_1_120_headless.sam
samtools view -H SAMPLE_PicardDedup_mapqfilt.bam > SAMPLE_PicardDedup_mapqfilt_header.sam
cat SAMPLE_PicardDedup_mapqfilt_header.sam SAMPLE_PicardDedup_mapqfilt_1_120_headless.sam > SAMPLE_PicardDedup_mapqfilt_1_120.sam
rm SAMPLE_PicardDedup_mapqfilt_1_120_headless.sam
```

In this step we're first having samtools view print the sam file without its header, which we're piping to awk, which filters for only fragments 120 bp or shorter. We then, however, need to put the header back on the file, so we print the header alone using samtools view -H, and we use cat to tack it back onto the filtered file.

## 7. Feeding our alignment file into HOMER for downstream analysis

```
module load homer/default
module load samtools/1.16.1

makeTagDirectory SAMPLE_1_120/ SAMPLE_PicardDedup_mapqfilt_1_120.sam
makeUCSCfile SAMPLE_1_120/ -o auto
```

Here, we're giving HOMER the final sam file, from which it will make a tag directory, a HOMER-specific format that it uses to feed into its many other commands (e.g., calling peaks, annotating peaks, finding motif enrichment, etc.). Then, the makeUCSCfile command will make a bedGraph file that you can use to look at your library coverage in a genome browser like IGV or UCSC.